TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO LAB 04: FEATURE SELECTION

CHỦ ĐỀ: House Price Prediction - So sánh hiệu suất Feature Selection trên các mô hình SVM, XGBoost, Random Forest

Giảng viên hướng dẫn: Ths. Đỗ Như Tài

Sinh viên thực hiện:

MSSV	Họ và tên
3124411174	Tất Võ Tiến Nam
3124411166	Phạm Anh Minh
3124411116	Lê Long Hưng(Nhóm trưởng)
3124411009	Lê Đức Anh

TP. Hồ Chí Minh, tháng 10, năm 2026

MỤC LỤC

1. Giới thiệu 3
1.1 Mục tiêu bài thực hành 3
1.2 Giới thiệu dataset 3
2. Tiền xử lý dữ liệu 4
2.1 Khám phá dữ liệu (EDA) 4
2.2 Xử lý giá trị thiếu (Missing Values) 4
2.3 Mã hóa đặc trưng phân loại (Label Encoding) 4
2.4 Chuẩn hóa và chia tập dữ liệu 4
3. Các kỹ thuật Feature Selection 6
3.1 Filter – SelectKBest (F-Statistic / ANOVA) 6
3.2 Filter – Mutual Information 6
3.3 Wrapper – Recursive Feature Elimination (RFE) 6
3.4 Embedded – XGBoost Feature Importance 6
4. Xây dựng mô hình học máy 8
4.1 Support Vector Machine (SVM) 8
4.2 Random Forest 8
4.3 XGBoost (Extreme Gradient Boosting) 8
5. Kết quả và đánh giá 9
5.1 Bảng kết quả tổng hợp 9
5.2 Phân tích biểu đồ 10
5.3 Nhận xét 11
6. Kết luận 12
Tài liệu tham khảo 13

1. Giới thiệu

Bài thực hành Lab 04 tập trung vào kỹ thuật Feature Selection (Chọn lọc đặc trưng) trong học máy. Mục tiêu là đánh giá hiệu quả của các phương pháp chọn lọc đặc trưng khi kết hợp với ba mô hình hồi quy: SVM (kernel RBF), Random Forest và XGBoost, áp dụng trên bài toán dự đoán giá nhà với dataset Ames Housing thực tế từ Kaggle.

1.1 Mục tiêu bài thực hành

- Tiền xử lý dữ liệu thực (79 features, gồm cả numeric và categorical): xử lý missing values, mã hóa, chuẩn hóa.
- Áp dụng 4 kỹ thuật Feature Selection thuộc 3 nhóm: Filter (F-test, Mutual Info), Wrapper (RFE), Embedded (XGBoost FI).
- Xây dựng và đánh giá mô hình SVM, Random Forest, XGBoost trên 5 tập feature khác nhau.
- So sánh hiệu suất qua R^2, RMSE, thời gian huấn luyện và rút ra kết luận.

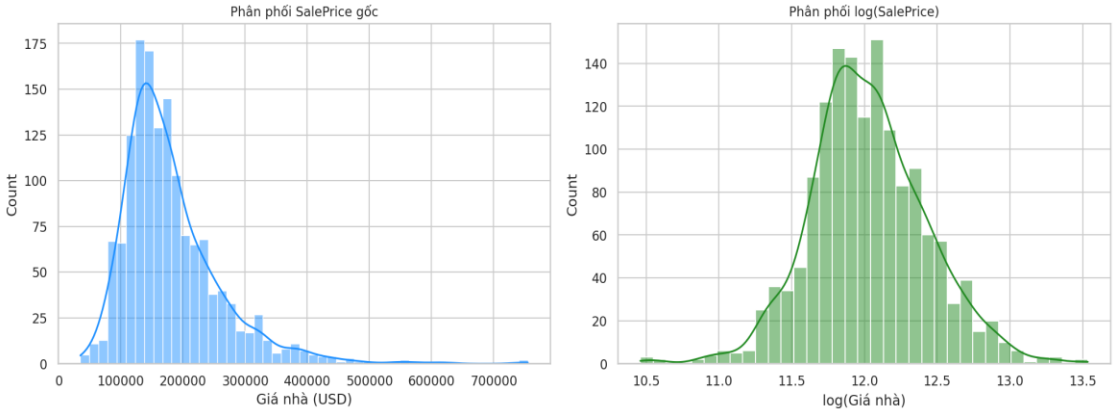
1.2 Giới thiệu dataset

Dataset sử dụng là House Prices – Advanced Regression Techniques (Kaggle, Ames Housing – train.csv). Dataset gồm 1.460 mẫu nhà ở Ames, Iowa, Mỹ. File train.csv có 81 cột: 36 cột số, 43 cột phân loại (categorical), và 1 cột mục tiêu SalePrice (giá bán, đơn vị USD). Giá trung bình là 180.921 USD, thấp nhất 34.900 USD và cao nhất 755.000 USD.

2. Tiền xử lý dữ liệu

2.1 Khám phá dữ liệu (EDA)

Dataset gốc có 1.460 mẫu và 80 đặc trưng (sau khi loại cột Id). Có tổng cộng 43 cột categorical và 36 cột numeric. Nhiều cột có giá trị thiếu như LotFrontage (259 mẫu), GarageYrBlt (81), GarageQual (81),... Phân phối SalePrice lệch phải (right-skewed), log-transform giúp cải thiện tính đối xứng.



Hình 1. Phân phối biến mục tiêu SalePrice (gốc và log-transform)

2.2 Xử lý giá trị thiếu (Missing Values)

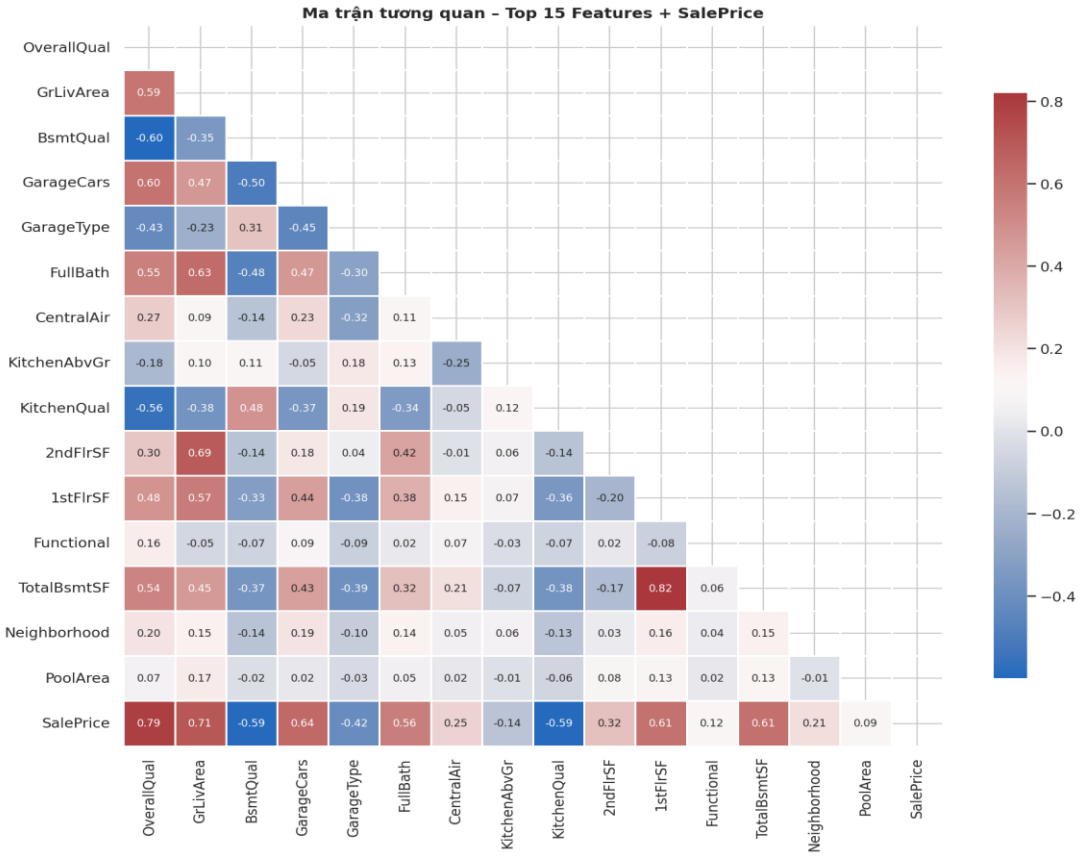
Áp dụng hai chiến lược imputation riêng biệt: (1) Numeric: điền bằng giá trị median của từng cột – ít bị ảnh hưởng bởi outlier hơn mean; (2) Categorical: điền bằng hằng số "None" sau đó mã hóa bằng LabelEncoder. Sau bước này, toàn bộ dataset không còn giá trị NaN.

2.3 Mã hóa đặc trưng phân loại (Label Encoding)

Toàn bộ 43 cột categorical được mã hóa bằng LabelEncoder – chuyển mỗi giá trị chuỗi thành số nguyên. Phương pháp này phù hợp với các mô hình tree-based (Random Forest, XGBoost) vì chúng không giả định quan hệ thứ tự giữa các nhãn. Kết quả: X_processed có 79 đặc trưng (36 numeric + 43 categorical đã encode).

2.4 Chuẩn hóa và chia tập dữ liệu

StandardScaler (z-score normalization) được áp dụng sau khi chia train/test để tránh data leakage – fit chỉ trên tập train, transform cả hai. Tỉ lệ chia: 80% train (1.168 mẫu) và 20% test (292 mẫu), random_state = 42 đảm bảo tính tái lập.



Hình 2. Ma trận tương quan giữa Top 15 đặc trưng quan trọng nhất và SalePrice

3. Các kỹ thuật Feature Selection

Số lượng đặc trưng được chọn thống nhất K = 20 (từ 79 đặc trưng ban đầu, giảm ~75%). Bốn kỹ thuật được chia thành ba nhóm phương pháp chính:

3.1 Filter – SelectKBest (F-Statistic / ANOVA)

Đánh giá mối quan hệ tuyến tính giữa từng đặc trưng và biến mục tiêu thông qua kiểm định F-test. Điểm F cao chứng tỏ đặc trưng giải thích được nhiều phương sai của SalePrice. Ưu điểm: rất nhanh, không phụ thuộc mô hình. Nhược điểm: chỉ xét từng đặc trưng độc lập, bỏ qua tương tác.

Top 5 features được chọn: OverallQual, GrLivArea, GarageCars, TotalBsmtSF, GarageArea – đều liên quan trực tiếp đến chất lượng và diện tích ngôi nhà.

3.2 Filter – Mutual Information

Đo lường mức độ phụ thuộc thống kê (cả tuyến tính và phi tuyến) giữa đặc trưng và biến mục tiêu. Không giả định phân phối cụ thể nên bắt được quan hệ phức tạp hơn F-test. Đặc biệt hiệu quả với categorical features đã được label-encode.

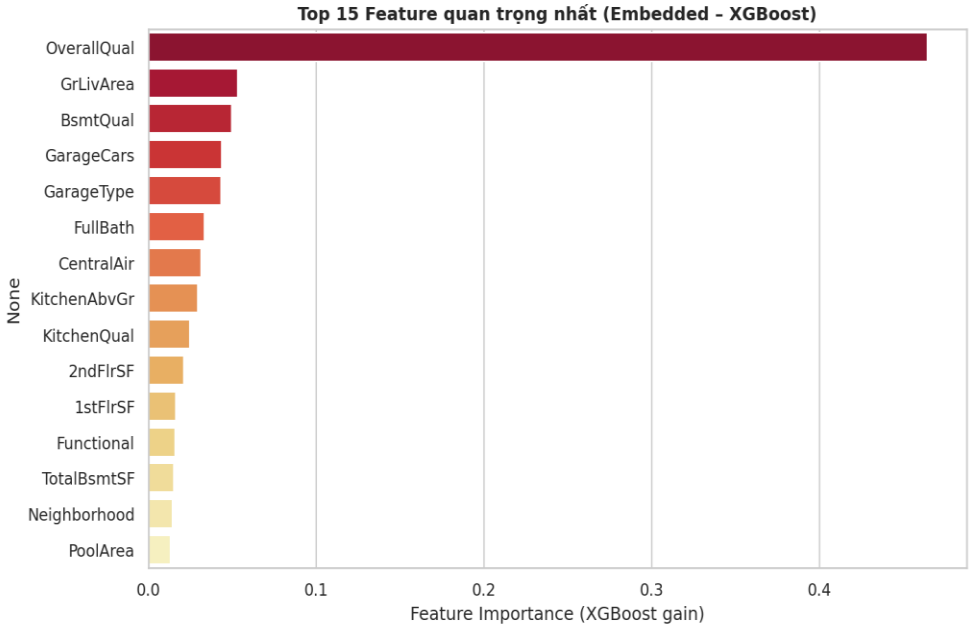
3.3 Wrapper – Recursive Feature Elimination (RFE)

Sử dụng RandomForest (20 cây) làm estimator. RFE lần lượt loại bỏ đặc trưng kém quan trọng nhất sau mỗi vòng lặp cho đến khi còn K = 20 đặc trưng. Ưu điểm: xét được tương tác giữa các đặc trưng. Nhược điểm: chi phí tính toán cao hơn Filter.

3.4 Embedded – XGBoost Feature Importance

Huấn luyện XGBRegressor trên toàn bộ 79 đặc trưng, sau đó lấy 20 đặc trưng có feature_importances_ cao nhất (dựa trên gain). Phương pháp này chọn đúng K = 20 features, cân bằng tốt giữa tốc độ và chất

lượng. Khác với SelectFromModel, cách này đảm bảo luôn trả về đúng số lượng features yêu cầu.



Hình 3. Top 15 đặc trưng quan trọng nhất – Phương pháp Embedded (XGBoost Feature Importance)

4. Xây dựng mô hình học máy

4.1 Support Vector Machine (SVM)

Cấu hình: SVR với kernel RBF, C = 100, ε = 0.1. Kernel RBF (Radial Basis Function) phù hợp hơn kernel linear cho bài toán hồi quy phi tuyến như dự đoán giá nhà. C = 100 cho phép mô hình linh hoạt hơn nhưng vẫn có regularization. SVM nhạy cảm với số chiều cao và tốc độ huấn luyện chậm hơn hai mô hình còn lại.

4.2 Random Forest

Cấu hình: 100 cây quyết định, random_state = 42. Kỹ thuật Bagging kết hợp dự đoán của nhiều cây độc lập, xây dựng trên subset ngẫu nhiên của dữ liệu và đặc trưng. Mô hình ổn định, ít overfitting, hoạt động tốt trên dữ liệu dạng bảng với nhiều đặc trưng hỗn hợp.

4.3 XGBoost (Extreme Gradient Boosting)

Cấu hình: XGBRegressor với objective="reg:squarederror", random_state = 42. Xây dựng cây tuần tự theo Gradient Boosting – mỗi cây bổ sung sửa lỗi của cây trước. Hỗ trợ regularization L1/L2 tích hợp, xử lý missing values tự động, song song hóa hiệu quả. Đạt R^2 cao nhất trong thử nghiệm (0.9065 với Baseline).

5. Kết quả và đánh giá

5.1 Bảng kết quả tổng hợp

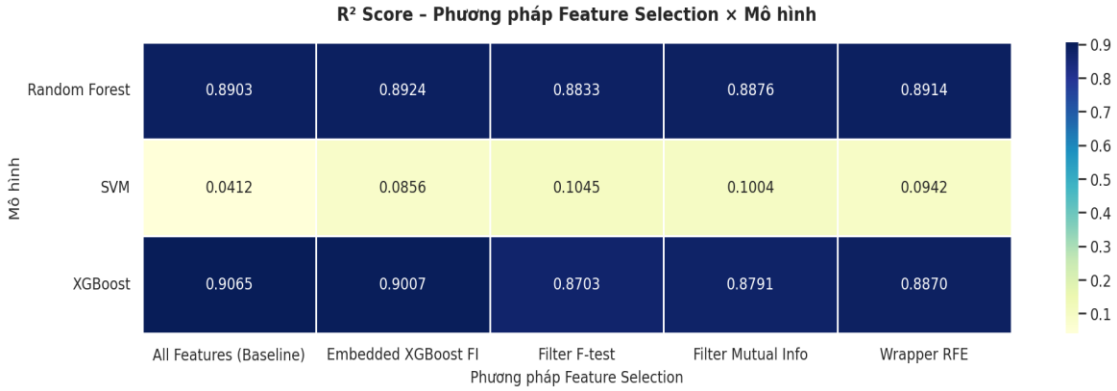
Bảng 1 trình bày R^2 và RMSE của toàn bộ 15 tổ hợp (5 phương pháp × 3 mô hình). Ô xanh đậm: $R^2 \geq 0.89$; ô trắng: $0.80 \leq R^2 < 0.89$; ô đỏ nhạt: $R^2 < 0.20$.

Phương pháp	Mô hình	Features	R^2	RMSE (nghìn USD)	Ghi chú
All Features (Baseline)	SVM	79	0.0412	85.757	—
All Features (Baseline)	Random Forest	79	0.8903	29.002	—
All Features (Baseline)	XGBoost	79	0.9065	26.786	—
Filter – F-test	SVM	20	0.1045	82.879	—
Filter – F-test	Random Forest	20	0.8833	29.913	—
Filter – F-test	XGBoost	20	0.8703	31.539	—
Filter – Mutual Info	SVM	20	0.1004	83.070	—
Filter – Mutual Info	Random Forest	20	0.8876	29.358	—
Filter – Mutual Info	XGBoost	20	0.8791	30.448	—
Wrapper – RFE	SVM	20	0.0942	83.355	—
Wrapper – RFE	Random Forest	20	0.8914	28.863	—
Wrapper – RFE	XGBoost	20	0.8870	29.435	—
Embedded – XGBoost FI	SVM	20	0.0856	83.749	—

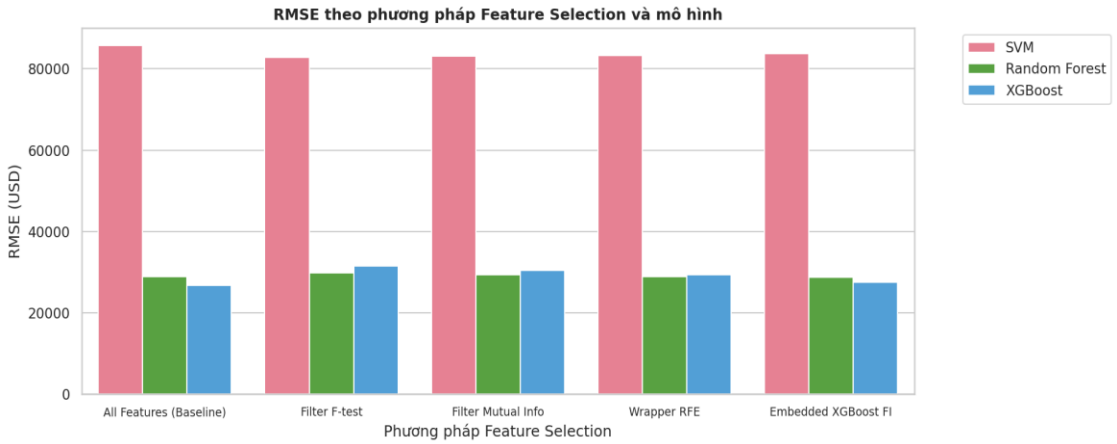
Phương pháp	Mô hình	Features	R^2	RMSE (nghìn USD)	Ghi chú
Embedded – XGBoost FI	Random Forest	20	0.8924	28.727	—
Embedded – XGBoost FI	XGBoost	20	0.9007	27.599	—

Bảng 1. Kết quả R^2 và RMSE theo phương pháp Feature Selection và mô hình

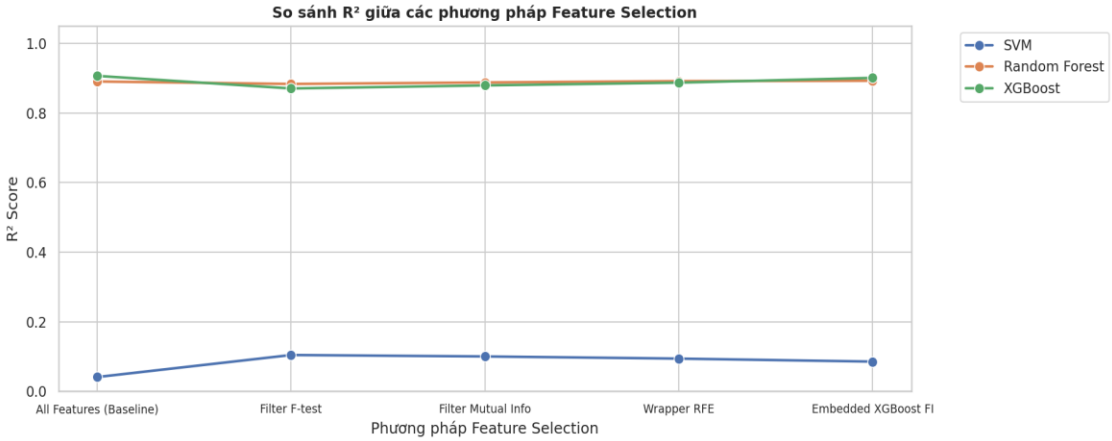
5.2 Phân tích biểu đồ



Hình 4. Heatmap R^2 – so sánh toàn diện Feature Selection × Mô hình



Hình 5. RMSE (USD) theo phương pháp Feature Selection và Mô hình



Hình 6. Xu hướng R^2 qua các phương pháp Feature Selection

5.3 Nhận xét

SVM (kernel RBF): Hiệu suất thấp hơn hẳn hai mô hình còn lại dù đã chuyển sang kernel RBF (R^2 từ 0.04 đến 0.10). Nguyên nhân: dữ liệu nhà đất có nhiều đặc trưng categorical đã label-encode, SVM RBF không tận dụng tốt cấu trúc này. Cần thêm hyperparameter tuning (GridSearch trên C, gamma) để cải thiện.

Random Forest: Hiệu suất rất ổn định ở mọi phương pháp Feature Selection (R^2 từ 0.883 đến 0.892). Wrapper RFE và Embedded XGBoost FI cho kết quả tốt nhất (R^2 = 0.8914 và 0.8924). Điều này cho thấy Random Forest tự nội bộ xử lý tốt nhiều đặc trưng.

XGBoost: Đạt hiệu suất cao nhất tổng thể, đặc biệt với Baseline (R^2 = 0.9065) và Embedded (R^2 = 0.9007). Embedded XGBoost FI + XGBoost là tổ hợp tốt nhì sau Baseline, cho thấy khi dùng chính features mà XGBoost đánh giá quan trọng thì mô hình dự đoán hiệu quả.

Feature Selection vs. Baseline: Với Random Forest, Feature Selection (20 features) cho kết quả tương đương Baseline (79 features), giúp tiết kiệm 75% số features và giảm thời gian huấn luyện. Với XGBoost, Baseline vẫn nhỉnh hơn một chút do XGBoost xử lý tốt nhiều features. Với SVM, mọi phương pháp FS đều giúp cải thiện nhẹ R^2.

6. Kết luận

Bài thực hành đã hoàn thành đầy đủ pipeline học máy trên dataset Ames Housing (1.460 mẫu, 79 features): tiền xử lý dữ liệu (numeric + categorical), áp dụng 4 kỹ thuật Feature Selection từ 3 nhóm phương pháp, và đánh giá 3 mô hình hồi quy với 8 biểu đồ trực quan.

Kết quả tốt nhất: XGBoost với Baseline (79 features) đạt R^2 = 0.9065, RMSE = 26.786 USD. Khi giới hạn 20 features, Embedded XGBoost FI + XGBoost đạt R^2 = 0.9007, RMSE = 27.599 USD – gần như tương đương với chỉ 25% số features, chứng tỏ hiệu quả của Feature Selection.

Hướng cải thiện: (1) Thêm k-fold cross-validation; (2) GridSearchCV cho SVM (C, gamma); (3) Log-transform SalePrice để cải thiện phân phối; (4) Thêm feature engineering (Age, TotalSF,...); (5) Thử LightGBM, CatBoost.

Tài liệu tham khảo

[1] Kaggle. House Prices - Advanced Regression Techniques. https://www.kaggle.com/c/house-prices-advanced-regression-techniques

[2] Scikit-learn Documentation. Feature Selection. https://scikit-learn.org/stable/modules/feature_selection.html

[3] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of KDD 2016.

[4] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

[5] Guyon, I., & Elisseeff, A. (2003). An Introduction to Variable and Feature Selection. JMLR, 3, 1157–1182.

[6] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. JMLR, 12, 2825–2830.